

CSA0609-DESIGN AND ANALYSIS OF ALGORITHMS

KRISHA MEENATCHI M (192511338)

Exp-07: Selection Sort on Random, Reverse and Sorted Arrays

Aim:

To implement Selection Sort on random, reverse and sorted arrays and analyze its time complexity.

Algorithm:

1. Read the size of the array.
2. Generate three types of arrays:
 - Random array
 - Reverse sorted array
 - Already sorted array
3. Apply Selection Sort to each array.
4. Find the minimum element in the unsorted portion.
5. Swap it with the first element of the unsorted portion.
6. Repeat until the array is sorted.
7. Display all three sorted arrays.
8. Display the complexity.

Source Code:

```
import random
```

```
n = int(input("Enter number of elements: "))
```

```
random_arr = [random.randint(1, 100) for _ in range(n)]
```

```
reverse_arr = list(range(n, 0, -1))
```

```
sorted_arr = list(range(1, n + 1))
```

```
def selection_sort(arr):
```

```
    for i in range(len(arr) - 1):
```

```
        min_index = i
```

```
        for j in range(i + 1, len(arr)):
```

```
            if arr[j] < arr[min_index]:
```

```
                min_index = j
```

```
    arr[i], arr[min_index] = arr[min_index], arr[i]
```

```
selection_sort(random_arr)
```

```
selection_sort(reverse_arr)
```

```
selection_sort(sorted_arr)
```

```
print("Sorted Random Array:", random_arr)
```

```
print("Sorted Reverse Array:", reverse_arr)
```

```
print("Sorted Sorted Array:", sorted_arr)
```

```
print("Time Complexity: O(n^2)")
```

```
print("Space Complexity: O(1)")
```

Output:

```
Enter number of elements: 5
Sorted Random Array: [14, 34, 74, 76, 93]
Sorted Reverse Array: [1, 2, 3, 4, 5]
Sorted Sorted Array: [1, 2, 3, 4, 5]
Time Complexity:  $O(n^2)$ 
Space Complexity:  $O(1)$ 
```

Note: The random array values can be different each time because they are generated randomly.

Result:

Selection Sort was successfully applied to random, reverse and sorted arrays, and its complexity was analyzed.